

02246 Mandatory Assignment

A01 - Temporal Logics*

To be submitted on DTU Learn - see deadline on DTU Learn

You are encouraged to work in groups, but you must clearly identify the contributions of each group member, and you will be jointly responsible for the finished report. Register your group on DTU Learn before submitting as group submission.

Answers to all parts should be typed up using LaTeX and submitted electronically as a PDF report using the provided template. Drawings and formulae may be handwritten and scanned. More detailed instructions as to the style of answer we expect for each part are included below.

Some tasks require to upload files.

*Thanks to Michael Smith (original author), and Lijun Zhang, Kebin Zeng and Flemming Nielson and Alberto Lluch Lafuente (contributors)

A01 - Temporal Logics

A01P: Practical Problems

A01P.1 - Mithra Iyengar Ramprabu For the FCFS scheduler provided (file `FCFS.nm`), we would like to verify that whenever a client has an active job, the scheduler has that job somewhere in its queue. For example, in the case of the first client, we require that whenever $state_1 = 1$, then either $job_1 = 1$ or $job_2 = 1$.

a) Express this as two CTL properties — one for each client.

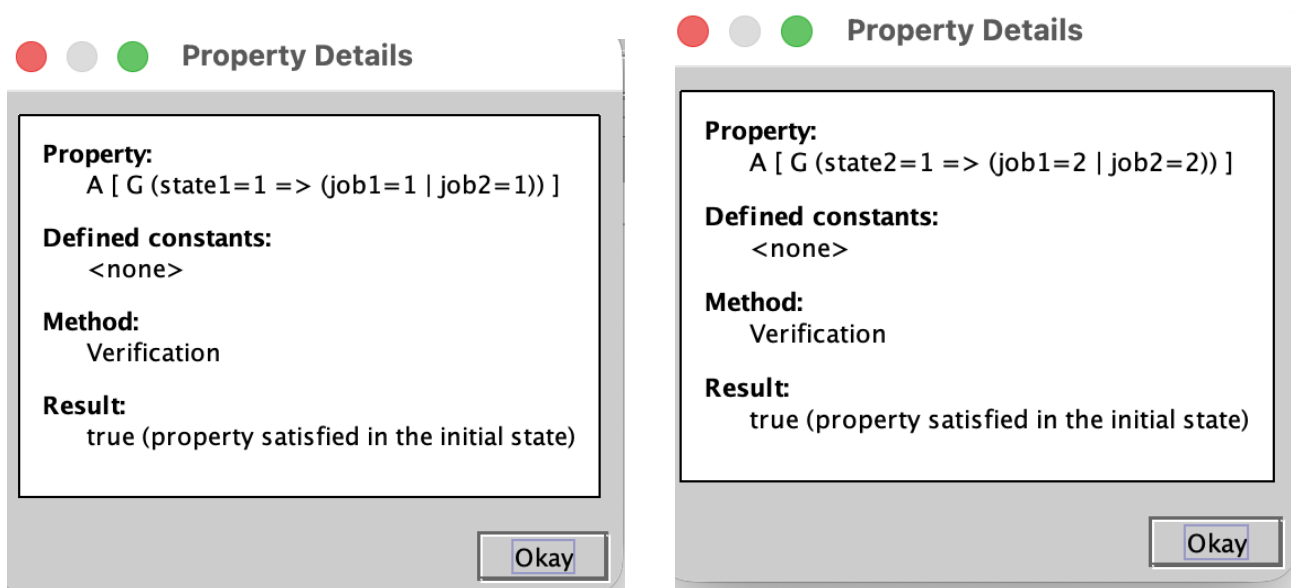
a) (a) $A[G (state_1 = 1 \Rightarrow (job_1 = 1 \mid job_2 = 1))]$

(b) $A[G (state_2 = 1 \Rightarrow (job_1 = 2 \mid job_2 = 2))]$

UPLOAD REQUIRED: a prism property file `A01P.1.a.props`.

b) Use PRISM to verify whether these properties hold in the FCFS scheduler model. Provide a screenshot showing that this is the case.

See Figure 1.



(a) Client 1's property.

(b) Client 2's property.

Figure 1: Verification results for the client properties on `FCFS.nm`.

c) Write down two similar properties for the SRT scheduler provided (file `SRT.nm`), explaining your construction.

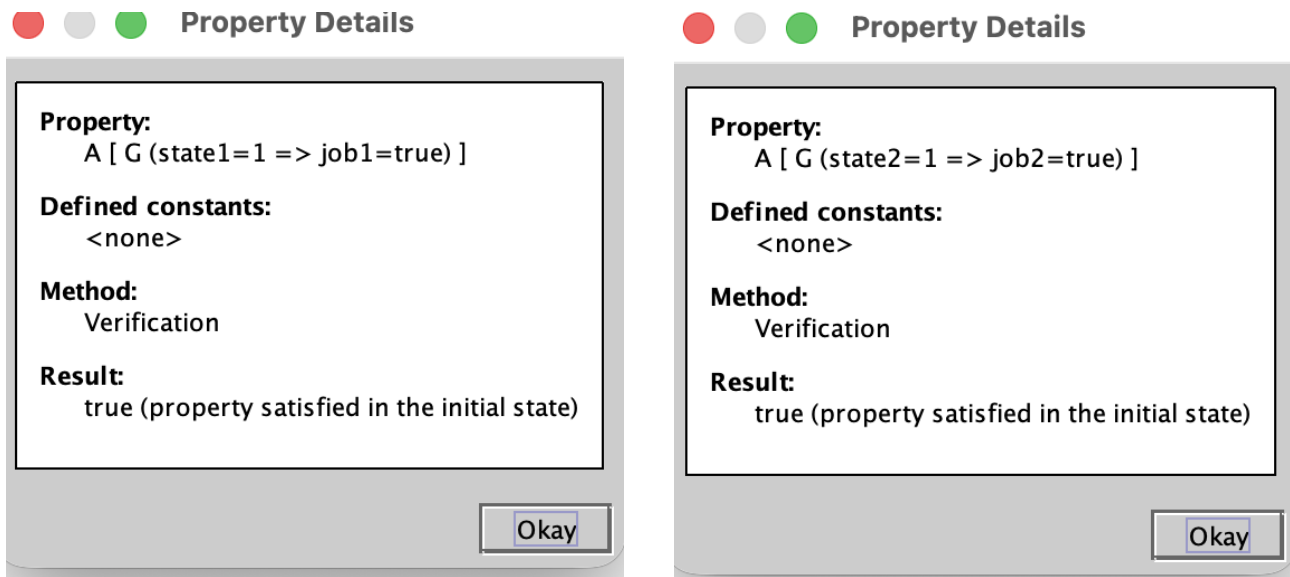
(a) $A[G (state1=1 \Rightarrow job1=true)]$

(b) $A[G (state2=1 \Rightarrow job2=true)]$

UPLOAD REQUIRED: a prism property file `A01P.1.c.props`.

d) Verify whether they hold in the model. Provide a screenshot showing the result.

See Figure 2.



(a) Client 1's property.

(b) Client 2's property.

Figure 2: Verification results for the client properties on `SRT.nm`.

A01P.2 - Mithra Iyengar Ramprabu Add another client to the PRISM model of the FCFS scheduler. You will need to modify the *Scheduler* module to cope with the extra client, but for now do not increase the length of the queue.

UPLOAD REQUIRED: a prism model file `A01P.2.prism`.

- a) Explain the changes that you made to the model, and argue why they satisfy the above instructions.

Two changes were made. First, a new module `client3`, a copy of `client1`, was added in the same way as `client2`; no new logic is needed since every client behaves the same. Secondly, `job1` and `job2` were widened from `[0..2]` to `[0..3]`, and matching `[create3]`, `[serve3]`, `[finish3]` commands were added to the scheduler, replicating the existing commands for clients 1 and 2. There are still exactly two queue slots (`job1`, `job2`) – only the range of values each slot can hold has increased to fit client 3's id, not the number of slots itself.

- b) How many reachable states are in the new model?

214 reachable states. See Figure 3

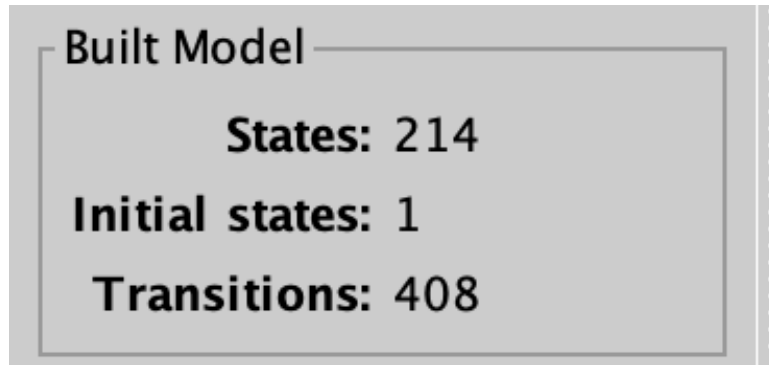


Figure 3: Number of reachable states with Client 3

- c) What will happen if the queue is full when a client attempts to create a job?

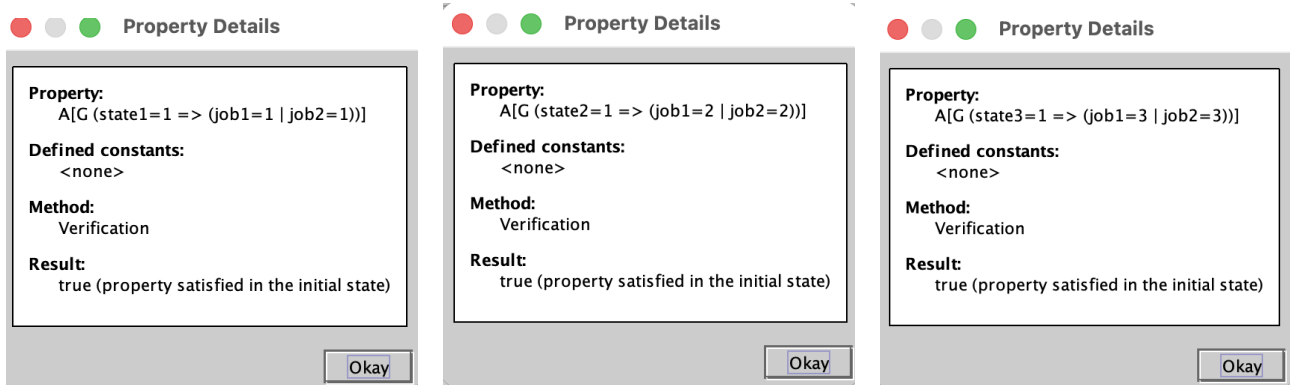
Nothing breaks; the client just has to wait. Starting a job needs two things to be true at the same time: the client must be free, and there must be an empty queue slot. If both slots are full, that second condition fails, so the "start a job" action simply can't happen yet. The client stays waiting until a slot opens up (once whoever's ahead of them finishes). This situation only arises with 3 client, with just 2 clients the queue could never actually fill up while someone was still waiting, since there were never more than 2 jobs to fit into 2 slots.

- d) Do the properties you have previously verified still hold in the model? If not, why not? Provide a screenshot showing the results.

Yes; all three properties hold, verified against all reachable state of the new model (Figures 4a, 4b, and 4c):

- $A[G \text{ (state1=1} \Rightarrow (\text{job1=1} \mid \text{job2=1}))]$
- $A[G \text{ (state2=1} \Rightarrow (\text{job1=2} \mid \text{job2=2}))]$
- $A[G \text{ (state3=1} \Rightarrow (\text{job1=3} \mid \text{job2=3}))]$ (the added property for client 3)

The moment a client starts a job, the model marks them as active and puts their job in the queue in the very same step; those two things always happen together, never one without the other. That doesn't change no matter how many clients there are or how busy the queue gets, so the property keeps holding.



(a) Client 1's property.

(b) Client 2's property.

(c) Client 3's property.

Figure 4: Verification results for the three client properties on the 3-client model.

A01P.3 - Loke Skjoldan Gødsbøll Jørgensen Now additionally modify the *Scheduler* module so that the queue is of length three.

UPLOAD REQUIRED: a prism model file A01P.3.prism.

- a) Explain the changes that you made to the model, and argue why they are correct.

The first change we make is to add a third slot in the scheduler's queue:

```
job3 : [0..3] init 0; // Third job in the queue
```

We also need to update our logic for placing jobs at the end of the queue, like so:

```
[create1] job3=0 -> (job3'=1);
[create2] job3=0 -> (job3'=2);
[create3] job3=0 -> (job3'=3);
```

And we need to allow the scheduler to shift jobs from slot 3 to slot 2:

```
[] job2=0 & job3>0 -> (job2'=job3) & (job3'=0);
```

- b) How many reachable states are in the new model?

According to PRISM, the new model has **1459** states.

- c) Do the properties now hold in the model? If not, why not? Provide a screenshot showing the results.

To verify that it still holds that whenever a client has an active job, the scheduler has that job somewhere in its queue, we need to modify our properties to account for the third queue slot:

- $A[G \text{ (state1=1} \Rightarrow (\text{job1=1} \mid \text{job2=1} \mid \text{job3=1}))]$
- $A[G \text{ (state2=1} \Rightarrow (\text{job1=2} \mid \text{job2=2} \mid \text{job3=2}))]$
- $A[G \text{ (state3=1} \Rightarrow (\text{job1=3} \mid \text{job2=3} \mid \text{job3=3}))]$

See Figure 5.

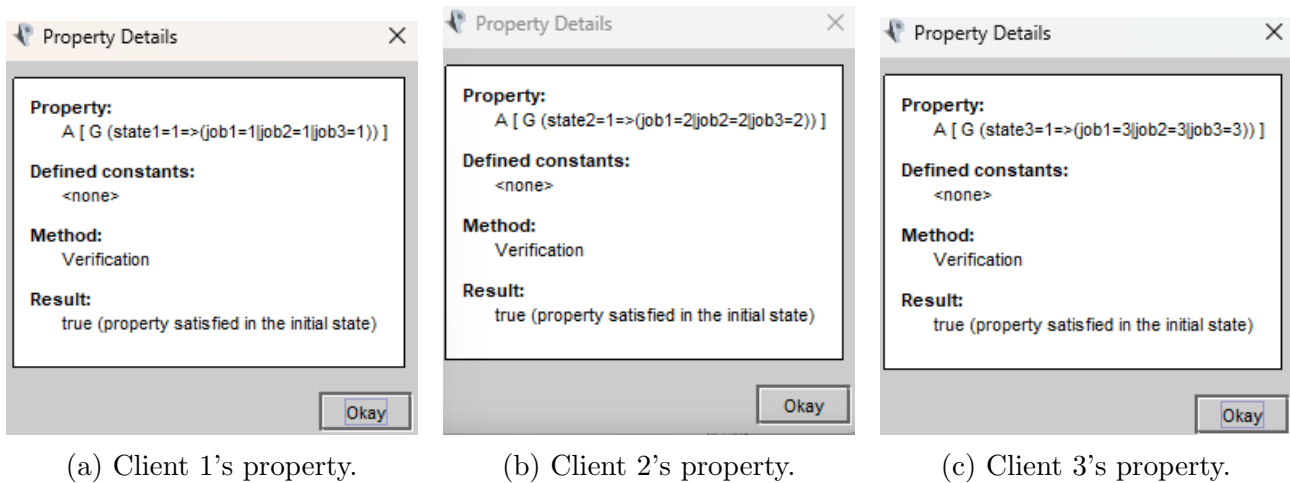


Figure 5: Verification results for the three client properties on the 3-client model with queue length three.

A01T: Theoretical Problems

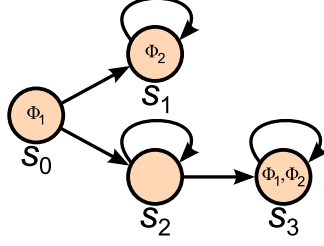


Figure 6: A transition system

A01T.1 - Stefano Giroletti Consider the transition system, shown graphically in Figure 6. The states are represented by circles, whose names are shown beneath them, and whose labels are shown inside them. The initial state is s_0 . Determine whether the following properties hold in state s_0 . You can encode the transition system in Figure 6 as a PRISM module, and you can use PRISM to check if your answers are correct but you have to explain why they hold or do not hold using the formal semantics of temporal logics.

a) $AX \Phi_2$.

Does not hold. Consider the path $\pi = s_0 s_2 (s_3)^\omega$. Then $\pi \in \text{paths}(s_0)$, but $\pi[1] \not\models \Phi_2$, so it is a counterexample.

b) $AF \Phi_2$.

Does not hold. Consider the path $\pi = s_0 s_2 (s_2)^\omega$. Then $\pi \in \text{paths}(s_0)$, but $\pi[i] \not\models \Phi_2$ for all $i \geq 0$. Therefore, by semantics, the property does not hold.

c) $EF \Phi_1$.

Holds. Consider the path $\pi = s_0 s_2 (s_3)^\omega$. Since $s_3 \models \Phi_1$, it follows by semantics that $s_0 \models EF \Phi_1$.

d) $F \Phi_1$.

Holds. Since every path starting from s_0 begins with s_0 and $s_0 \models \Phi_1$, then for all $\pi \in \text{paths}(s_0)$ it holds that $\pi[0] \models \Phi_1$.

e) $A[\Phi_1 U \Phi_2]$.

Does not hold. Consider the path $\pi = s_0 s_2 (s_2)^\omega \in \text{paths}(s_0)$. Since $s_0, s_2 \not\models \Phi_2$, then $\nexists i \in \mathbb{N}$ such that $\pi[i] \models \Phi_2$. By semantics, it follows that $\pi \not\models \Phi_1 U \Phi_2$, making it a counterexample.

f) $G(\neg \Phi_1 \rightarrow F \Phi_2)$.

Does not hold. Consider the path $\pi = s_0 (s_2)^\omega \in \text{paths}(s_0)$. We start by showing that $\pi[1] \not\models \neg \Phi_1 \rightarrow F \Phi_2$. Call π' the suffix starting at index 1, which is $(s_2)^\omega$. We know that $s_2 \models \neg \Phi_1$, but $\nexists j \in \mathbb{N}. \pi'[j] \models \Phi_2$. This means $\pi' \not\models F \Phi_2$, and thus $\pi' \not\models \neg \Phi_1 \rightarrow F \Phi_2$. Therefore, the entire path π does not satisfy $G(\neg \Phi_1 \rightarrow F \Phi_2)$.

g) $G(\neg \Phi_1 \rightarrow EF \Phi_2)$.

Holds. Since it has the global quantifier, we need to analyze every possible state in every possible execution. First, we can say that it trivially holds in s_0, s_3 . Then, we need to consider states s_1, s_2 :

- For $s_1 \models \neg\Phi_1$, consider $\pi_1 = (s_1)^\omega$. Then $\exists i \in \mathbb{N}. \pi_1[i] \models \Phi_2$, namely $i = 0$.
- For $s_2 \models \neg\Phi_1$, consider $\pi_2 = s_2(s_3)^\omega$. Then $\exists i \in \mathbb{N}. \pi_2[i] \models \Phi_2$, namely $i = 1$.

For all states s reachable from s_0 , it holds that $s \models \neg\Phi_1 \rightarrow EF\Phi_2$. It follows that $\forall \pi \in \text{paths}(s_0). \forall i \in \mathbb{N}. \pi[i] \models \neg\Phi_1 \rightarrow EF\Phi_2$, meaning $s_0 \models G(\neg\Phi_1 \rightarrow EF\Phi_2)$.

A01T.2 - Stefano Giroletti For each of the following pairs of CTL formulae, determine whether (a) they are equivalent, (b) one implies the other, or (c) neither implies the other. Explain your reasoning.

HINT: If one formula does not imply another one, you should be able to find a transition system witnessing this. Try to make it as small as possible. Otherwise, you will have to refer to the formal semantics to support your answer.

a) $EX\ EF\ \Phi$ and $EF\ EX\ \Phi$.

They are equivalent. Let T be a transition system with initial state s_0 .

($\Rightarrow$) $s_0 \models EX\ EF\ \Phi \implies \exists \pi \in paths(s_0)$ s.t. $\pi[1] \models EF\ \Phi$. Thus, there exists a state s_k at depth $k \geq 1$ along π where $s_k \models \Phi$. This means the preceding state s_{k-1} satisfies $EX\ \Phi$. Since s_{k-1} is reachable from s_0 , it follows that $s_0 \models EF\ EX\ \Phi$.

($\Leftarrow$) $s_0 \models EF\ EX\ \Phi \implies \exists \pi \in paths(s_0)$ reaching a state s_k ($k \geq 0$) where $s_k \models EX\ \Phi$. Thus, s_k has a successor s_{k+1} satisfying Φ . The execution $s_0 \dots s_k \rightarrow s_{k+1}$ shows that from the immediate successor $\pi[1]$, Φ is eventually reachable. Therefore, $s_0 \models EX\ EF\ \Phi$.

b) $AX\ AF\ \Phi$ and $AF\ AX\ \Phi$.

$AF\ AX\ \Phi \implies AX\ AF\ \Phi$, but the converse does not hold.

($\Rightarrow$) Assume $s_0 \models AF\ AX\ \Phi$. Any path from s_0 eventually reaches a state s_k where all its successors satisfy Φ . Take any immediate successor s_1 of s_0 : any path starting from s_1 is a suffix of a path from s_0 , and thus must eventually satisfy Φ (either at s_k or its successor). Since s_1 is arbitrary, $s_0 \models AX\ AF\ \Phi$.

($\nRightarrow$) See Figure 7 for the counterexample. $s_0 \models AX\ AF\ \Phi$ because all paths from its successors eventually satisfy Φ . However, $s_0 \not\models AF\ AX\ \Phi$ because there exists a path where we can indefinitely delay reaching a state whose successors *all* satisfy Φ .

c) $AG\ EF\ \Phi$ and $EF\ AG\ \Phi$.

Neither implies the other.

($\nRightarrow$) $AG\ EF\ \Phi \not\implies EF\ AG\ \Phi$: In Figure 8, $s_0 \models AG\ EF\ \Phi$ because Φ is eventually reachable from any state. However, $s_0 \not\models EF\ AG\ \Phi$ because no reachable state can guarantee Φ globally along all its future paths (the system can always return to s_0).

($\nRightarrow$) $EF\ AG\ \Phi \not\implies AG\ EF\ \Phi$: In Figure 9, $s_0 \models EF\ AG\ \Phi$ because s_1 is reachable and satisfies $AG\ \Phi$. However, $s_0 \not\models AG\ EF\ \Phi$ because there is a path leading to s_2 , a trap state where Φ is false forever.

d) $AG\ (\Phi_1 \wedge \Phi_2)$ and $(AG\ \Phi_1) \wedge (AG\ \Phi_2)$.

They are equivalent.

($\Rightarrow$) $s_0 \models AG(\Phi_1 \wedge \Phi_2) \implies$ on all paths, at all states, $\Phi_1 \wedge \Phi_2$ holds. This trivially means Φ_1 holds everywhere on all paths ($AG\ \Phi_1$) and Φ_2 holds everywhere on all paths ($AG\ \Phi_2$). Hence, $(AG\ \Phi_1) \wedge (AG\ \Phi_2)$ holds.

($\Leftarrow$) $s_0 \models (AG\ \Phi_1) \wedge (AG\ \Phi_2) \implies$ on all paths, Φ_1 is always true, and Φ_2 is always true. Thus, at any given state on any path, both Φ_1 and Φ_2 are true simultaneously, which means $s_0 \models AG(\Phi_1 \wedge \Phi_2)$.

e) $EF (\Phi_1 \wedge \Phi_2)$ and $(EF \Phi_1) \wedge (EF \Phi_2)$.

$EF(\Phi_1 \wedge \Phi_2) \implies (EF \Phi_1) \wedge (EF \Phi_2)$, but the converse does not hold.

$(\implies)$ $s_0 \models EF(\Phi_1 \wedge \Phi_2) \implies$ there is a path reaching a state where both Φ_1 and Φ_2 hold simultaneously. This inherently provides a path reaching Φ_1 ($EF \Phi_1$) and a path reaching Φ_2 ($EF \Phi_2$).

$(\nRightarrow)$ See Figure 10 for the counterexample. $s_0 \models EF \Phi_1$ (via s_1) and $s_0 \models EF \Phi_2$ (via s_2). However, $s_0 \not\models EF(\Phi_1 \wedge \Phi_2)$ since no single state reachable from s_0 satisfies both Φ_1 and Φ_2 at the same time.

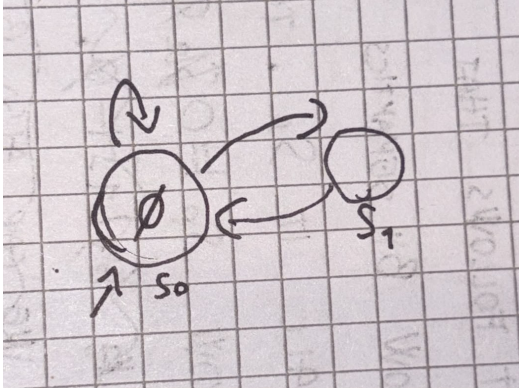


Figure 7: $AX AF \Phi \not\implies AF AX \Phi$

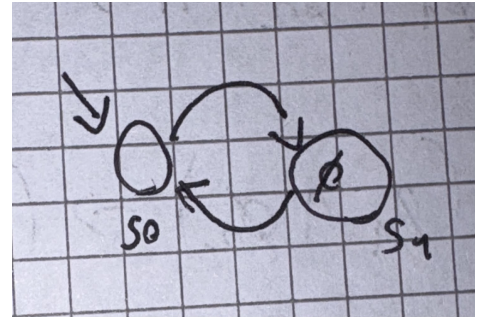


Figure 8: $AG EF \Phi \not\implies EF AG \Phi$

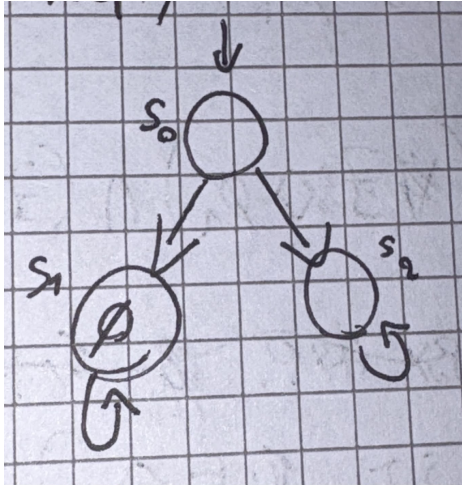


Figure 9: $EF AG \Phi \not\implies AG EF \Phi$

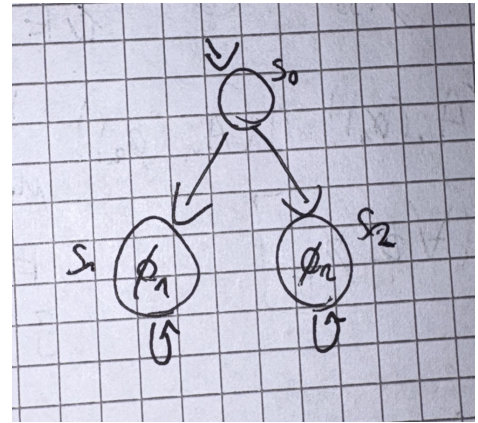


Figure 10: $(EF \Phi_1) \wedge (EF \Phi_2) \not\implies EF(\Phi_1 \wedge \Phi_2)$

A01T.3 - Maria Mitsi For each of the following LTL formulae, find a small-as-possible transition system that satisfies it.

a) XFp .

To have a smallest transitions system we need two states s_0 and s_1 :

$$s_0 \longrightarrow s_1, \quad s_1 \longrightarrow s_1,$$

where p holds in s_1 and does not hold in s_0 .

The only path is $s_0, s_1, s_1, s_1, \dots$ and after one step from the initial state p eventually holds. Hence XFp is satisfied.

b) $\neg FXp$.

A smallest transition system consists of one state s with a self-loop:

$$s \longrightarrow s,$$

where p is false in s .

The only path is $s, s, s, \dots$ and therefore Xp is false at every position.

Consequently, FXp is false, and hence $\neg FXp$ is satisfied.

c) $(XFp) \wedge (\neg FXp)$.

No transition system can satisfy this formula.

XFp requires that after the initial state there is eventually a state in which p holds. Thus, p must hold at some position $i > 0$ on the path.

However, if p holds at position $i > 0$, then at the preceding position $i - 1$ we have Xp . Therefore, FXp is true, contradicting the second conjunct $\neg FXp$.

Hence the two requirements are mutually inconsistent, and there is no transition system satisfying $(XFp) \wedge (\neg FXp)$.

d) GFp .

A smallest transition system consists of one state s with a self-loop:

$$s \longrightarrow s,$$

where p holds in s .

The only path is $s, s, s, \dots$ so p holds infinitely often. Therefore, GFp is satisfied.

e) $\neg FGp$.

Again we have a transition system with one state s with a self-loop:

$$s \longrightarrow s,$$

where p is false in s .

The only path is $s, s, s, \dots$ and p never holds. Thus, Gp is false at every position, so FGp is false.

f) $(GFp) \wedge (\neg FGp)$.

The transition system has two states s_p and $s_{\neg p}$:

$$s_p \longrightarrow s_{\neg p}, \quad s_{\neg p} \longrightarrow s_p,$$

where p holds in s_p and does not hold in $s_{\neg p}$.

The only path is $s_p, s_{\neg p}, s_p, s_{\neg p}, \dots$

Thus, p occurs infinitely often, so GFp is satisfied. At the same time, p does not remain true, since it is false in every occurrence of $s_{\neg p}$. Hence FGp is false and therefore $\neg FGp$ is true.

A02T.4 - Maria Mitsi Write down a CTL* formula for each of the following properties, which are described in natural language. Try to exploit the features of CTL* as much as possible (i.e. try to avoid inserting quantifiers before each temporal operator as in CTL). In each case, argue whether or not the property can also be expressed in CTL with the same meaning as the your CTL* proposal.

- a) There is a path on which Φ holds infinitely often.

$$E[GF\Phi]$$

This cannot be expressed in CTL, since CTL cannot require the same path to satisfy $GF\Phi$.

- b) For all paths, Φ_1 holds along the path until Φ_2 holds in some state s and Φ_3 holds in the state that immediately follows s .

$$A[\Phi_1 U (\Phi_2 \wedge X\Phi_3)]$$

This can also be expressed in CTL as

$$A[\Phi_1 U (\Phi_2 \wedge AX\Phi_3)].$$

- c) There is a path on which either Φ_1 eventually holds or Φ_2 eventually holds.

$$E[F\Phi_1 \vee F\Phi_2]$$

This can also be expressed in CTL as

$$EF\Phi_1 \vee EF\Phi_2.$$

- d) For all paths, either Φ_1 always holds or Φ_2 always holds.

$$A[G\Phi_1 \vee G\Phi_2]$$

This cannot be expressed in CTL with the same meaning, since

$$AG\Phi_1 \vee AG\Phi_2$$

requires the same property to hold on all paths.